

# Core concepts

The central interface in the Spring Data repository abstraction is `Repository`. It takes the domain class to manage as well as the identifier type of the domain class as type arguments. This interface acts primarily as a marker interface to capture the types to work with and to help you to discover interfaces that extend this one.

## TIP

Spring Data considers domain types to be entities, more specifically aggregates. So you will see the term "entity" used throughout the documentation that can be interchanged with the term "domain type" or "aggregate".

As you might have noticed in the introduction it already hinted towards domain-driven concepts. We consider domain objects in the sense of DDD. Domain objects have identifiers (otherwise these would be identity-less value objects), and we somehow need to refer to identifiers when working with certain patterns to access data. Referring to identifiers will become more meaningful as we talk about repositories and query methods.

The [CrudRepository](#) and [ListCrudRepository](#) interfaces provide sophisticated CRUD functionality for the entity class that is being managed.

## CrudRepository Interface

```
public interface CrudRepository<T, ID> extends Repository<T, ID> {  
  
    <S extends T> S save(S entity);           1  
  
    Optional<T> findById(ID primaryKey);      2  
  
    Iterable<T> findAll();                    3  
  
    long count();                             4  
  
    void delete(T entity);                    5  
  
    boolean existsById(ID primaryKey);        6  
  
    // ... more functionality omitted.  
}
```

JAVA

- 1 Saves the given entity.
- 2 Returns the entity identified by the given ID.
- 3 Returns all entities.
- 4 Returns the number of entities.
- 5 Deletes the given entity.
- 6 Indicates whether an entity with the given ID exists.

The methods declared in this interface are commonly referred to as CRUD methods.

`ListCrudRepository` offers equivalent methods, but they return `List` where the `CrudRepository` methods return an `Iterable`.

#### | IMPORTANT

The repository interface implies a few reserved methods like `findById(ID identifier)` that target the domain type identifier property regardless of its property name. Read more about this in [“Defining Query Methods”](#).

You can annotate your query method with `@Query` to provide a custom query if a property named `Id` doesn't refer to the identifier. Following that path can easily lead to confusion and is discouraged as you will quickly hit type limits if the `ID` type and the type of your `Id` property deviate.

#### | NOTE

We also provide persistence technology-specific abstractions, such as `JpaRepository` or `MongoRepository`. Those interfaces extend `CrudRepository` and expose the capabilities of the underlying persistence technology in addition to the rather generic persistence technology-agnostic interfaces such as `CrudRepository`.

Additional to the `CrudRepository`, there are [PagingAndSortingRepository](#) and [ListPagingAndSortingRepository](#) which add additional methods to ease paginated access to entities:

*PagingAndSortingRepository interface*

```
public interface PagingAndSortingRepository<T, ID> {  
  
    Iterable<T> findAll(Sort sort);  
  
    Page<T> findAll(Pageable pageable);  
}
```

JAVA

#### | NOTE

Extension interfaces are subject to be supported by the actual store module. While this documentation explains the general scheme, make sure that your store module supports the interfaces that you want to use.

To access the second page of `User` by a page size of 20, you could do something like the following:

JAVA

```
PagingAndSortingRepository<User, Long> repository = // ... get access to a bean
Page<User> users = repository.findAll(PageRequest.of(1, 20));
```

`ListPagingAndSortingRepository` offers equivalent methods, but returns a `List` where the `PagingAndSortingRepository` methods return an `Iterable`.

In addition to pagination, scrolling provides a more fine-grained access to iterate through chunks of larger result sets.

In addition to query methods, query derivation for both count and delete queries is available. The following list shows the interface definition for a derived count query:

#### Derived Count Query

JAVA

```
interface UserRepository extends CrudRepository<User, Long> {

    long countByLastname(String lastname);

}
```

The following listing shows the interface definition for a derived delete query:

#### Derived Delete Query

JAVA

```
interface UserRepository extends CrudRepository<User, Long> {

    long deleteByLastname(String lastname);

    List<User> removeByLastname(String lastname);

}
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and

copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.